

- Description

This module manages a smart traffic flow monitoring and adaptive signal control system, ensuring that:

1. **Traffic density at intersections is continuously monitored to optimize signal timing.**
2. **If traffic congestion exceeds a predefined threshold, green light duration is extended to improve flow.**
3. **If traffic is light, green light duration is reduced to improve overall efficiency.**
4. **A log of all traffic conditions, signal adjustments, and efficiency improvements is maintained for urban planning.**

This system helps reduce congestion, improve traffic efficiency, and optimize signal timing dynamically.

- Specification

module SmartTrafficSignalControl

var currentTrafficDensity, maxTrafficDensity, minTrafficDensity, greenLightDuration: integer

var log: sequence of string

process updateTrafficDensity(newDensity: integer)

functional scenario form:

currentTrafficDensity <= maxTrafficDensity && log = log $\oplus$ ("Traffic Flow Optimal: " + currentTrafficDensity + " vehicles."), Traffic flow optimized, green light duration adjusted.

||

currentTrafficDensity > maxTrafficDensity && log = log $\oplus$ ("High Congestion Warning: " + currentTrafficDensity + " vehicles, Extending Green Light Duration."), High congestion warning triggered, green light extended.

||

currentTrafficDensity < minTrafficDensity && log = log $\oplus$ ("Low Traffic Info: " + currentTrafficDensity + " vehicles, Reducing Green Light Duration."), Low traffic detected, green light reduced.

end process

end module

- Java Code

```
package code;
import java.util.*;
class SmartTrafficSignal {
    private int maxTrafficDensity;
    private int minTrafficDensity;
    private int currentTrafficDensity;
    private int greenLightDuration;
    private List<String> log;
```

```

    public SmartTrafficSignal(int maxTrafficDensity, int minTrafficDensity, int initialTrafficDensity, int initialGreenLightDuration) {
        this.maxTrafficDensity = maxTrafficDensity;
        this.minTrafficDensity = minTrafficDensity;
        this.currentTrafficDensity = initialTrafficDensity;
        this.greenLightDuration = initialGreenLightDuration;
        this.log = new ArrayList<>();
    }

    public void updateTrafficDensity(int newDensity) {
        currentTrafficDensity = newDensity;
        adjustSignalTiming();
        String message = String.format("Current Traffic Density: %d vehicles | Green Light: %d seconds",
                                        currentTrafficDensity, greenLightDuration);
        if (currentTrafficDensity > maxTrafficDensity) {
            message += " | WARNING: High Congestion! Extending Green Light Duration.";
        } else if (currentTrafficDensity < minTrafficDensity) {
            message += " | INFO: Low Traffic! Reducing Green Light Duration.";
        } else {
            message += " | Traffic Flow is Optimal.";
        }
        System.out.println(message);
        log.add(message);
    }

    private void adjustSignalTiming() {
        if (currentTrafficDensity > maxTrafficDensity) {
            greenLightDuration += 10;
        } else if (currentTrafficDensity < minTrafficDensity) {
            greenLightDuration = Math.max(10, greenLightDuration - 5);
        }
    }

    public void printLog() {
        System.out.println("\nTraffic Signal Adjustment Log:");
        for (String logEntry : log) {
            System.out.println(logEntry);
        }
    }
}

public class c53_SmartTrafficSystem {
    public static void main(String[] args) {

```

```

SmartTrafficSignal signal = new SmartTrafficSignal(50, 10, 30,
30);

// testcase-VT:
signal.updateTrafficDensity(60);
signal.updateTrafficDensity(40);
signal.updateTrafficDensity(5);
signal.updateTrafficDensity(20);
signal.updateTrafficDensity(70);
signal.printLog();

// testcase-FT:
//     signal.updateTrafficDensity(36);
//     signal.updateTrafficDensity(40);
//     signal.updateTrafficDensity(10280);
//     signal.updateTrafficDensity(10446);
//     signal.updateTrafficDensity(189);
//     signal.printLog();

// testcase-Z3:
//     signal.updateTrafficDensity(60);
//     signal.updateTrafficDensity(40);
//     signal.updateTrafficDensity(5);
//     signal.updateTrafficDensity(20);
//     signal.updateTrafficDensity(70);
//     signal.printLog();

// testcase-UVT:
signal.updateTrafficDensity(60);
signal.updateTrafficDensity(40);
signal.updateTrafficDensity(5);
signal.updateTrafficDensity(20);
signal.updateTrafficDensity(70);
signal.printLog();
}
}

```

- Coverage report

1. Test case number: 5
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	1.2 %	10	832	842
✓ src	1.2 %	10	832	842
✓ code	1.2 %	10	832	842
✓ c53_SmartTrafficSystem.java	100.0 %	10	0	10
> c53_SmartTrafficSystem		0	0	0
> SmartTrafficSignal	100.0 %	10	0	10

3. Vibration testing instruction coverage: 97.9%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	0.7 %	143	20,068	20,211
✓ src	0.7 %	143	20,068	20,211
✓ code	0.7 %	143	20,068	20,211
✓ c53_SmartTrafficSystem.java	97.9 %	143	3	146
> c53_SmartTrafficSystem	89.7 %	26	3	29
> SmartTrafficSignal	100.0 %	117	0	117

4. Fuzz testing branch coverage: 80%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	1.0 %	8	834	842
✓ src	1.0 %	8	834	842
✓ code	1.0 %	8	834	842
✓ c53_SmartTrafficSystem.java	80.0 %	8	2	10
> c53_SmartTrafficSystem		0	0	0
> SmartTrafficSignal	80.0 %	8	2	10

5. Fuzz testing instruction coverage: 89.7%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	0.6 %	131	20,080	20,211
✓ src	0.6 %	131	20,080	20,211
✓ code	0.6 %	131	20,080	20,211
✓ c53_SmartTrafficSystem.java	89.7 %	131	15	146
> c53_SmartTrafficSystem	89.7 %	26	3	29
> SmartTrafficSignal	89.7 %	105	12	117

6. Z3 branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	1.2 %	10	832	842
✓ src	1.2 %	10	832	842
✓ code	1.2 %	10	832	842
✓ c53_SmartTrafficSystem.java	100.0 %	10	0	10
> c53_SmartTrafficSystem		0	0	0
> SmartTrafficSignal	100.0 %	10	0	10

7. Z3 instruction coverage: 97.9%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	0.7 %	143	20,068	20,211
✓ src	0.7 %	143	20,068	20,211
✓ code	0.7 %	143	20,068	20,211
✓ c53_SmartTrafficSystem.java	97.9 %	143	3	146
> c53_SmartTrafficSystem	89.7 %	26	3	29
> SmartTrafficSignal	100.0 %	117	0	117

8. Unimproved V-method branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	1.2 %	10	832	842
✓ src	1.2 %	10	832	842
✓ code	1.2 %	10	832	842
✓ c53_SmartTrafficSystem.java	100.0 %	10	0	10
> c53_SmartTrafficSystem		0	0	0
> SmartTrafficSignal	100.0 %	10	0	10

9. Unimproved V-method instruction coverage: 97.9%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	0.7 %	143	20,068	20,211
✓ src	0.7 %	143	20,068	20,211
✓ code	0.7 %	143	20,068	20,211
✓ c53_SmartTrafficSystem.java	97.9 %	143	3	146
> c53_SmartTrafficSystem	89.7 %	26	3	29
> SmartTrafficSignal	100.0 %	117	0	117